Lab Assignment 2

Due Apr. 30, 2026 at 23:59:59

1 Objective

The purpose of this lab is to extend Lab 1's tiled dense matrix multiplication to a **batched** setting using **shared memory** . Given two batches of matrices A of shape (batch, M, K) and B of shape (batch, K, N), you will implement a CUDA kernel that computes C[b] = A[b] * B[b] for every b, exposing **batch-level parallelism** through the z-dimension of the CUDA grid.

2 Instructions

The code template in `template.cu` provides a starting point and handles the import and export as well as the checking of the solution. Students are expected to insert their code demarcated with `//@@`. Students are expected to leave the other code unchanged.

Edit the skeleton code to perform the following:

- **Extract batch, M, K, N from the 4x1 dims vector (input2.raw)**
- **Allocate host and device memory for the batched matrices**
- **Copy host memory to device**
- **Initialize 3D grid and 2D block dimensions (use blockIdx.z for batch)**
- **Invoke the batched CUDA kernel**
- **Copy results from device to host**
- **Free device memory**
- **Write the batched tiled CUDA kernel using shared memory**

Compile the template with the provided `Makefile`. The executable generated as a result of compilation can be run using the following command:

```
./BatchedMatMul_template -e <expected.raw> \
    -i <input0.raw>,<input1.raw>,<input2.raw> \
    -o <output.raw> -t matrix
```

where `<expected.raw>` is the expected output, `<input0.raw>` and `<input1.raw>` are the stacked A and B matrices, `<input2.raw>` is the 4x1 dims vector (batch, M, K, N) stored with header `4 1`, and `<output.raw>` is an optional path to store the results.

`README.md` has details on how to build `libgputk`, `template.cu`, and the dataset generator.

Input format reminder. The dataset generator writes the batch/M/K/N dimensions into `input2.raw` as a 4x1 column vector (header line `4 1`). Reading `input2.raw` with `gpuTKImport(..., &rows, &cols)` returns rows=4, cols=1. Cast the four float entries to int: `data[0]=batch, data[1]=M, data[2]=K, data[3]=N`.

3 What to Turn in

Submit a `report.pdf` with your `template.cu` as compressed `20xx-xxxxx.tar.gz` file. Your report should include the following:

1. How many floating point operations are being performed by your batched kernel?
2. How many global memory reads are being performed by your kernel?
3. How many global memory writes are being performed by your kernel?
4. Describe what further optimizations can be implemented to your kernel to achieve a performance speedup.
5. Explanation about your version of `template.cu`.
6. Execution times of the kernel on the correctness datasets (0-8) generated by the dataset generator (in a table or graph). Please include the system information where you performed your evaluation. For time measurement, use `gpuTKTime_start` and `gpuTKTime_stop` functions (details in `libgputk/README.md`).
7. Batch-level parallelism experiment. The dataset generator also produces datasets 9-14 with fixed per-batch size 64x64x64 and batch = 1, 2, 4, 8, 16, 32. Measure kernel execution time on each of these six datasets and report batch size vs kernel time (table or graph). Discuss how the batched kernel (using `blockIdx.z`) scales with batch count and at what point the GPU saturates.